

PisoFire M2

**High-Demand Sneaker
E-commerce Platform**

Group D

António Alberto 114622

Diogo Fernandes 114137

João Varela 113780





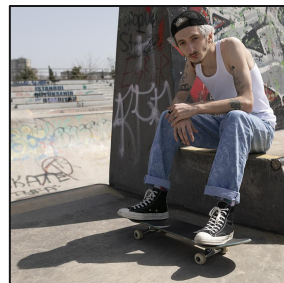
What is PisoFire?

- **Product** : Cloud-native e-commerce platform dedicated to premium sneaker releases.
- **Industry Domain** : High-volume retail focusing on "Flash Sales" .
- **SRE Challenge**: The system must operate reliably during periods of constant traffic; maintain availability and data integrity during “Flash Sales”.

Critical Path

Flash Sales Buying Path

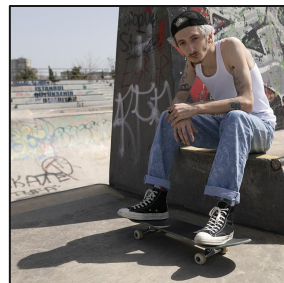
- User goes to flash sale landing page after launching time
- User can view the product's details
- User selects a size
- User adds item to cart
- User goes to checkout
- Payment is processed
- Inventory is decreased



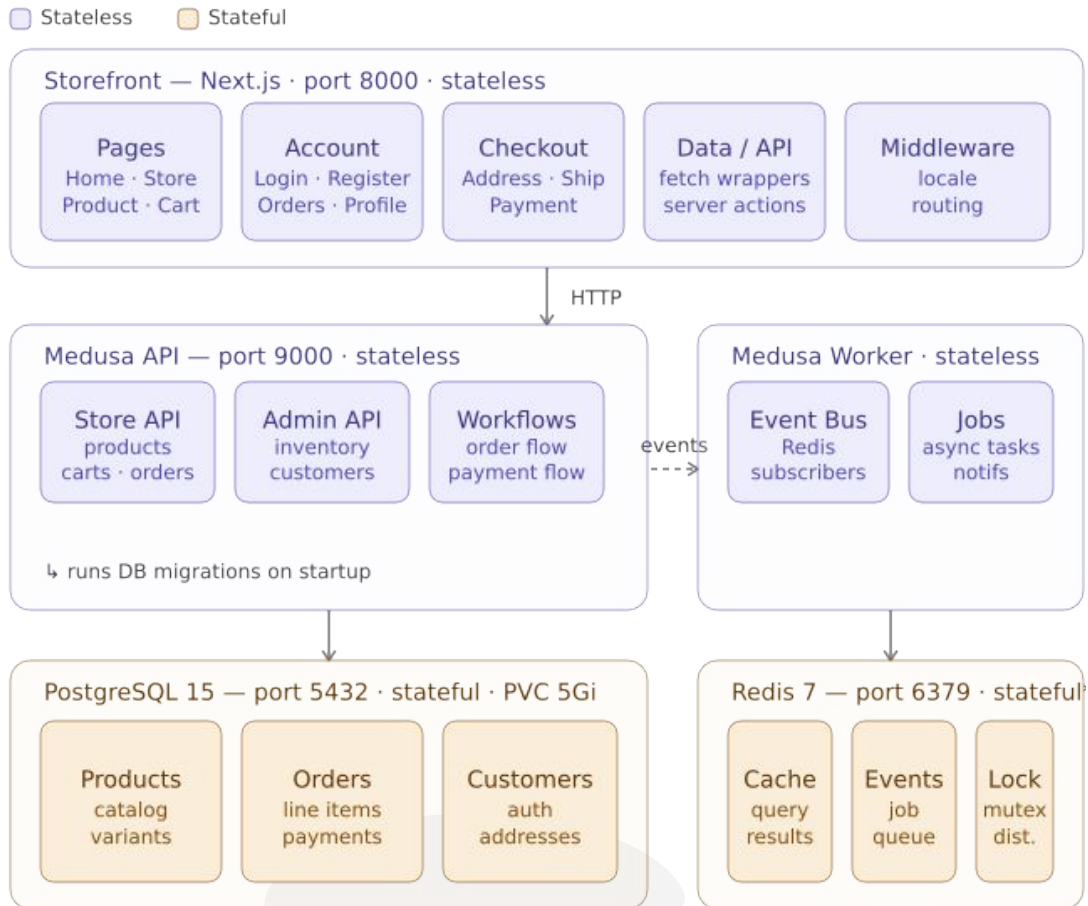
Critical Path

Flash Sales Buying Path

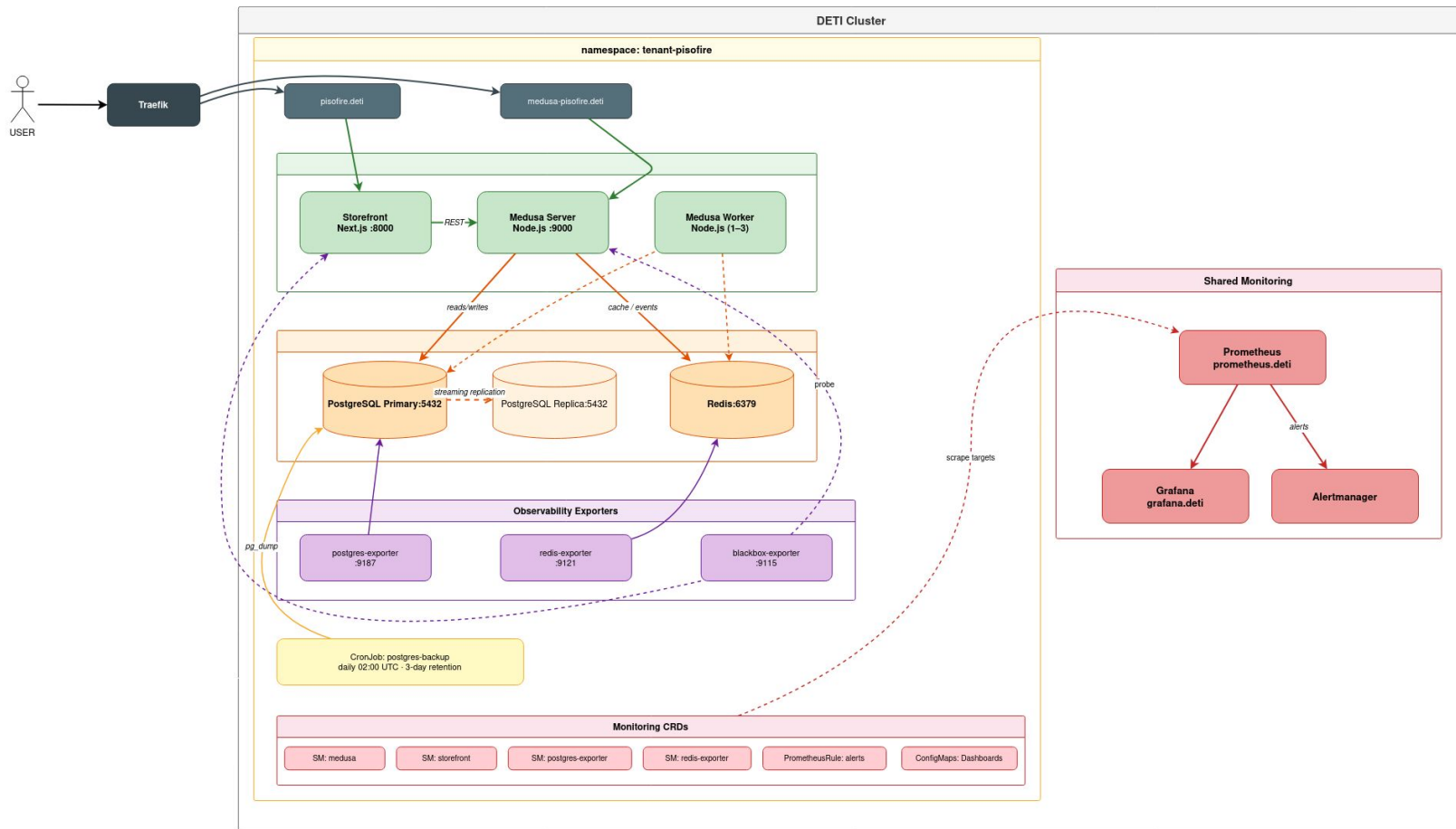
- GET /store/products/:id ← product page
- POST /store/carts ← add to cart
- POST /store/payment-* ← payment collection
- POST /store/orders ← order creation



M1 Architecture



M2 Architecture



Strategy

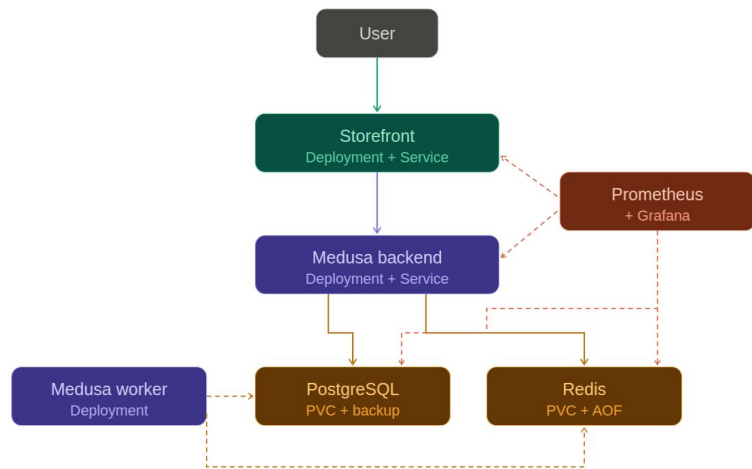
- Deploy to DETI Kubernetes using a reproducible workflow
- Build and tag Docker images for Medusa and Storefront
- Manage Kubernetes resources with Kustomize overlays
- Automate deploy, rollout checks and rollback with one script

```
./scripts/smoke-test-k8s.sh
```



Kubernetes Component Mapping

Product Component	Kubernetes resources
Storefront	Deployment, Service, HPA, PDB
Medusa Backend	Deployment, Service, HPA, PDB
Medusa worker	Deployment, HPA, PDB
PostgreSQL	Deployment, Service, PVC, PDB, Backup CronJob
Redis	Deployment, Service, PVC, PDB
Bootstrap	Jobs for store data and admin use
Observability	Metrics endpoints, ServiceMonitor, Grafana, Alerts

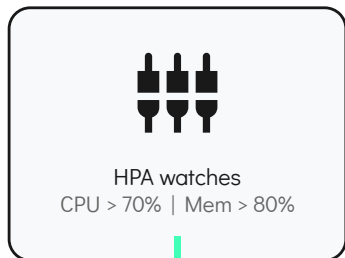


Health Checks

Component	Probe Method	Checks
Postgres	pg_isready / SQL query	Database accepts connections and can answer queries
Redis	redis-cli ping	Redis is reachable and responds to commands
Medusa API	/health + /readyz	Process is alive, database query works, Redis responds
Medusa Worker	/health	Worker process is alive
Storefront	/api/health + /api/ready	Process is alive and Medusa backend is ready

Autoscaling with HPA


Rare sneaker
drop



Before



2 Storefront

2 Medusa

After (Scaled)



5 Storefront

5 Medusa

```
Starting HPA scale demonstration
Namespace: tenant-pisofire
Target deployment: storefront
HPA: storefront
Initial replicas: 2
Temporarily lowering HPA targets to 1% for a deterministic demo.
Starting in-cluster HTTP load: workers=2 duration=120s
HPA storefront: desired=2 ready=2 cpu=3% memory=28%
HPA storefront: desired=2 ready=2 cpu=2% memory=29%
HPA storefront: desired=5 ready=2 cpu=43% memory=32%
HPA scale test passed
Namespace: tenant-pisofire
Target deployment: storefront
HPA: storefront
Initial replicas: 2
Scale-up observed: yes
Pinning HPA to 2 replicas for cleanup
Restoring deployment/storefront replica count to 2
HPA cooldown: cpu=unknown%/70% memory=unknown%/80%
Restoring HPA targets for storefront
```

Smoke Tests

The smoke test:

- Waits for Pods and Deployments
- Checks health endpoints
- Checks metrics endpoints
- Validates Store API flow
- Creates cart and order

```
➤ → PisoFire git:(master) ./scripts/smoke-test-k8s.sh
Waiting for core deployments in namespace pisofire
Waiting for deployment rollouts to finish
Waiting for bootstrap jobs completion
Checking autoscaling and disruption controls
Starting local port-forwards
Checking backend and storefront health/readiness
Checking Medusa admin authentication
Running end-to-end store API journey
Checking service DNS from inside the cluster
pod "pisofire-service-smoke-1779935854" deleted from pisofire namespace
Smoke test passed
Context: k3d-pisofire
Namespace: pisofire
Backend health: 200
Backend readiness: 200
Store readiness: 200
Backend metrics: 200
Storefront health: 200
Storefront readiness: 200
Storefront metrics: 200
Autoscaling controls: present
PostgreSQL backup controls: present
Stateful disruption controls: present
Redis persistence: present
Admin authentication: passed
Storefront page: 307
Service DNS checks: passed
Region: reg_01KS3007J1RK3XY44409P0SYV3
Product: prod_01KS30082MK6W1F3JEDWSEZJC2
Variant: variant_01KS300862A4GV7GFG1EEXETKLW
Cart: cart_01KSP73J28QVKTAZW6S5HTGKD
Shipping option: so_01KS3007VWEMZSV8GTMQ
Payment provider: pp_system_default
Payment collection: pay_col_01KSP73K3Z1F
Order: order_01KSP73K77NG4ZDQC2TW09R9VR
```

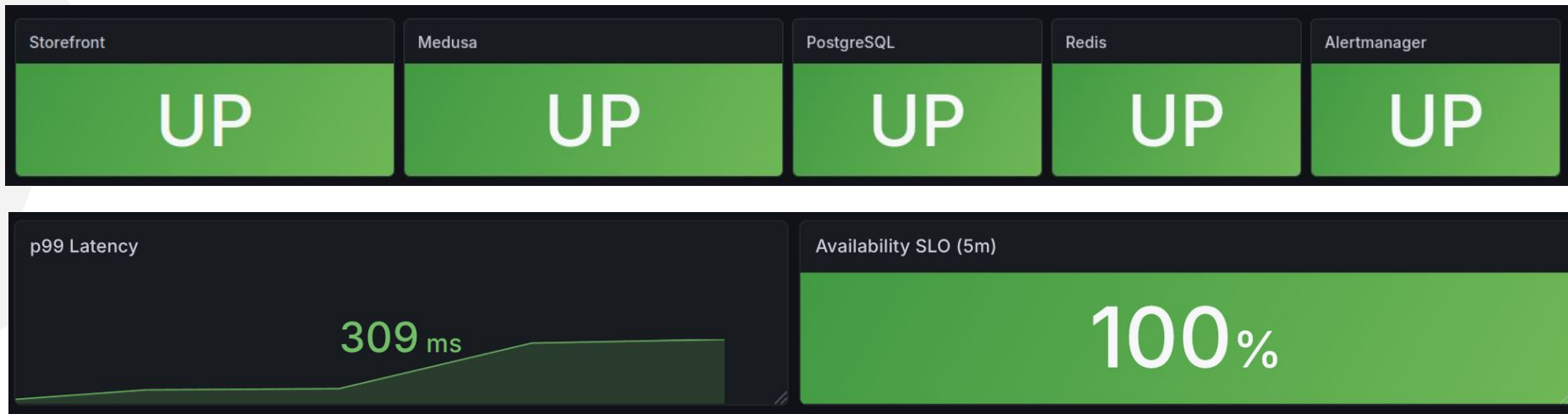
SLIs & SLOs

Type	SLI	SLO
Availability	% of non-5xx responses over 5 min	$\geq 99\%$
Reachability	blackbox probe success	$\geq 99\%$ probe success
Latency	p99 request duration	< 1 second
PostgreSQL Availability	pg_up	$\geq 99\%$
Redis Availability	redis_up	$\geq 99\%$
Checkout Availability	ratio of successful requests on the buying path	$\geq 99\%$
Checkout Latency	p99 response time on cart and payment endpoints	< 2 seconds

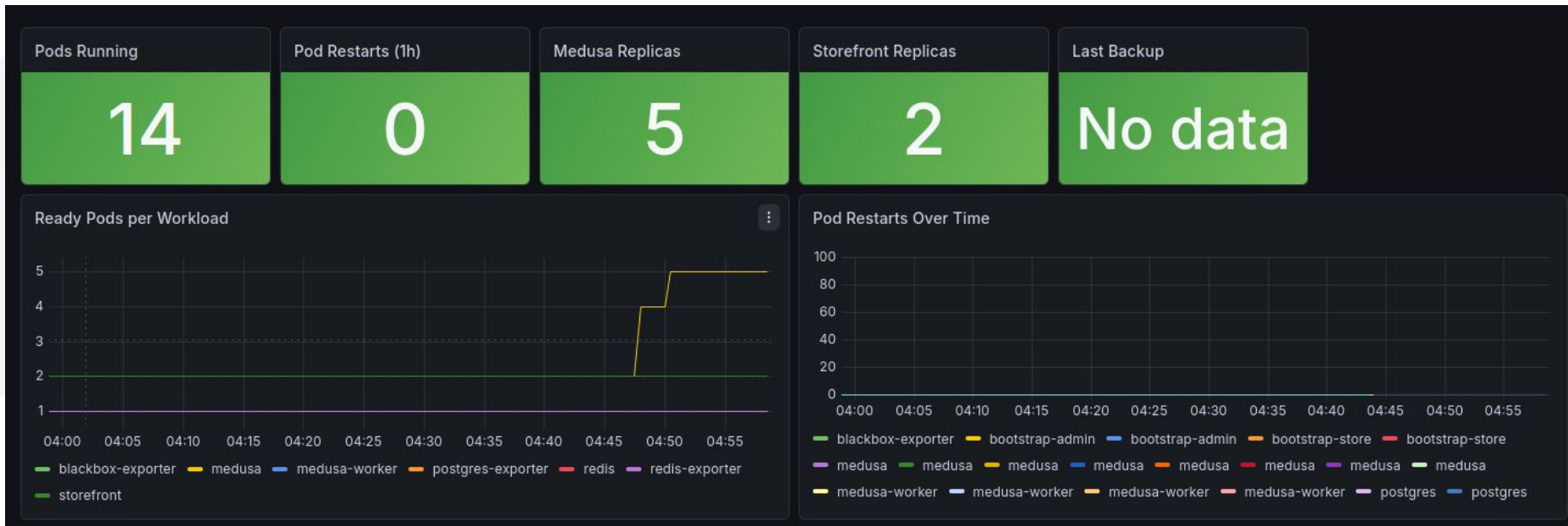
Alerting

Alert	Severity	Fires after	Condition
PisofireMedusaDown	critical	1 min	Prometheus can't scrape Medusa /metrics
PisofireHighErrorRate	warning	2 min	5xx rate > 1% of all requests (5m window)
PisofireHighLatency	warning	2 min	p95 latency > 1s (5m window)
PisofirePostgresDown	critical	1 min	postgres-exporter scrape target unreachable
PisofirePostgresHighConnections	warning	5 min	Total active connections > 80
PisofirePostgresBackupFailed	critical	5 min	Backup CronJob has a failed job
PisofirePostgresBackupStale	warning	15 min	Last successful backup > 26h ago
PisofireRedisDown	critical	1 min	redis-exporter scrape target unreachable
PisofireRedisHighMemory	warning	5 min	Redis memory > 85% of max

Observability - Services & Availability



Observability - Pod Status



Observability - Critical Path



Limitations and Conclusions

Bad Assumptions

- One database is enough
- Port-only health checks are enough
- Manual deploy steps are acceptable
- Local success means cluster readiness

Current Limitations

- Redis is single-instance
- No automatic DB failover
- Not supporting rolling updates
- No schema-version readiness yet



Thanks for your attention!

Any feedback among us?

